

ATHEERA · Stage 9: Agentic Forecasting Agent

Edge Case Testing Guide

Document Type	Edge Case Testing Guide — Engineering Reference
Scope	16 cases across 5 categories: Data Quality, Boundary Values, Business Logic, Error Handling, Seed & DB
Total Cases	16 (P1: 5 · P2: 8 · P3: 3)
Already Covered	14 cases already tested — see Section 01. This guide covers only the gaps.
Version	1.0 April 2026
Classification	Confidential — Engineering Only

01 Master Index

All 16 gap cases by priority. Cases already covered by the existing 386 tests are listed separately.

Gap cases — not yet tested (this document)

ID	Title	Category	Priority	Affects
DQ-01	All-zero demand series	Data Quality	P1	NaiveForecast, SESModel, HoltLinearTrend
DQ-03	Duplicate dates in time series	Data Quality	P1	ProphetModel (crash), Sub-Stage 9.2 promo lookup
BL-01	Declining demand — clamp to zero	Business Logic	P1	HoltLinearTrend.predict(), predict_all_horizons()
EH-01	All HP configs raise ModelFitError	Error Handling	P1	run_hp_tuning() in sub_stage_93.py
EH-02	Bare Exception from handler in run()	Error Handling	P1	run() in state_machine.py
BV-01	calibration_update_rate = 0.0	Boundary Values	P2	TenantParams.update() in tenant_params.py
BV-04	SESModel with smoothing_level = 0.0	Boundary Values	P2	SESModel in ses.py
BV-05	SES / Holt with smoothing_level = 1.0	Boundary Values	P2	SESModel, HoltLinearTrend
BV-06	HoltLinearTrend with smoothing_trend = 0.0	Boundary Values	P2	HoltLinearTrend in holt.py
BL-02	Croston TSB extinction pattern	Business Logic	P2	CrostonMethod._fit_tsb() in croston.py
BL-03	All optional features fail reliability floor	Business Logic	P2	run_feature_engineering() in sub_stage_92.py
DQ-02	Single-row DataFrame	Data Quality	P2	All models, Sub-Stage 9.3 train/test split
EH-03	TenantParams.load() returns 0 rows	Error Handling	P2	TenantParams.load() in tenant_params.py
SD-01	seed_tenant_params() override with 0.0	Seed & DB	P2	seed_tenant_params() in seed.py

ID	Title	Category	Priority	Affects
BV-02	calibration_update_rate = 1.0	Boundary Values	P3	TenantParams.update() in tenant_params.py
BV-03	TenantParams.update() evidence outside [0,1]	Boundary Values	P3	TenantParams.update() in tenant_params.py

Already covered by existing test suite (386 tests) — for reference only

Case	Location
NaN / Inf in qty column — all models	test_models.py
Empty DataFrame (0 rows) — NaiveForecast	test_models.py
Sparse DataFrame (< 14 rows) — all Sub-Stage 9.3 tests	test_sub_stages.py
Croston with 0 or 1 non-zero events → SES fallback	test_models.py (D2/D3)
Prophet with constant series (E002)	test_models.py
B2B weekend filter on a weekend-only seller (E006)	test_sub_stages.py
Redis double-acquire blocked, wrong token rejected	verify_run_lock.py
State machine invalid transitions, concurrent run() calls	test_state_machine.py
Bootstrap ordering invariant $p_{50} \leq p_{80} \leq p_{90}$ — 100 random seeds	test_base_bootstrap.py
Bootstrap with 0 / 1 / 2 residuals (E004)	test_base_bootstrap.py
UnknownParamError not inheriting KeyError	test_tenant_params.py

02 Data Quality Cases

Unusual but realistic input data that could arrive from upstream pipelines

DQ · Data Quality

These cases arise from real data problems: unlisted products, ETL bugs, new SKUs with sparse history.

DQ-01 All-Zero Demand Series

P1 — Highest Risk · Fix before production

Affects: NaiveForecast, SESModel, HoltLinearTrend (CrostonMethod already handled)

What happens

A SKU that was listed but never sold, or one that was delisted with trailing zeros, produces a series where every qty = 0.0 across the entire history. This is common for placeholder products, recently onboarded tenants, and items discontinued mid-season.

RISK

NaiveForecast computes a rolling mean of zeros — returns 0.0 silently (probably fine but unverified). SESModel initialises level from the first value (0.0) and never updates. HoltLinearTrend initialises both level and trend from the series — zero-sum may produce a degenerate or slightly negative trend. predict() in all models must clamp to ≥ 0 , but this clamp has not been verified against a zero-level series specifically.

EXPECTED BEHAVIOUR

All three models must produce a forecast of 0.0 across all horizons without crashing. No negative values. No NaN or Inf. CrostonMethod is exempt — its E001 fallback already handles this by falling back to SES.

Test Sketch

```
@pytest.fixture
def df_all_zeros():
    return pd.DataFrame({
        'date': pd.date_range('2024-01-01', periods=60),
        'qty': np.zeros(60),
    })

@pytest.mark.parametrize('Cls', [NaiveForecast, SESModel, HoltLinearTrend])
def test_all_zero_demand_produces_non_negative_forecast(Cls, df_all_zeros):
    m = Cls(hp=Cls({}).default_hp)
    m.fit(df_all_zeros, ['date', 'qty'])
    result = m.predict(df_all_zeros, ['date', 'qty'], horizon=30)
    assert np.all(result >= 0)
    assert np.all(np.isfinite(result))
```

DQ-02 Single-Row DataFrame

P2 — Medium Risk · Fix before stable release

Affects: All models, Sub-Stage 9.3 train/test split, compute_residuals()

What happens

A brand-new SKU added to the catalogue has exactly 1 day of sales recorded before the nightly run triggers. This is normal on the day a product is first listed.

RISK

Models computing rolling windows (NaiveForecast lag_periods=7) have no window to roll over. The train/test split in Sub-Stage 9.3 requires at least 14 rows — 1 row should hit the short-df guard and use default_hp. compute_residuals() slices df[-30:] returning a 1-element array — bootstrap_quantiles() then sees 1 residual and hits the E004 log-normal proxy path. This chain must complete without error.

EXPECTED BEHAVIOUR

Every model must complete fit() and predict_all_horizons() without crashing. All 8 string horizon keys must be present in the result. Sub-Stage 9.3 must detect the short-df condition and skip Thompson, using default_hp.

Test Sketch

```
@pytest.fixture
def df_one_row():
    return pd.DataFrame({'date': [pd.Timestamp('2024-01-01')], 'qty': [10.0]})

@pytest.mark.parametrize('Cls', [NaiveForecast, SESModel, CrostonMethod, HoltLinearTrend])
def test_single_row_does_not_crash(Cls, df_one_row):
```

```
m = Cls(hp=Cls({}).default_hp)
m.fit(df_one_row, ['date', 'qty'])
result = m.predict_all_horizons(df_one_row, ['date', 'qty'])
assert set(result.keys()) == EXPECTED_HORIZON_KEYS
```

DQ-03 Duplicate Dates in Time Series

P1 — Highest Risk · Fix before production

Affects: ProphetModel (hard crash), Sub-Stage 9.2 promo capping (double-cap), a

What happens

An upstream ETL bug delivers two rows for the same date — common with correction rows in inventory systems where a sale is reversed and re-recorded on the same day.

RISK	EXPECTED BEHAVIOUR
Prophet requires unique ds values. Duplicate dates raise ValueError inside Prophet's validation. If this is not caught and re-raised as ModelFitError, the raw exception propagates unchecked past the Sub-Stage 9.3 error handler. Sub-Stage 9.2 promo capping: two rows for the same promo date both get capped — the second cap is harmless but the first row's sample_weight may be duplicated if Prophet is the assigned model.	Prophet must catch the ValueError from duplicate dates and re-raise it as ModelFitError. Sub-Stage 9.3 then catches ModelFitError and applies the E005 fallback to Naive Forecast. Other models (Naive, SES, Holt, Croston) silently double-count the duplicate — they must not crash, even if the result is a slightly inflated level estimate.

Test Sketch

```
def test_prophet_duplicate_dates_raises_model_fit_error():
    dates = list(pd.date_range('2024-01-01', periods=29))
    dates += [pd.Timestamp('2024-01-15')] # duplicate
    df = pd.DataFrame({'date': dates, 'qty': np.ones(30) * 5.0})
    m = ProphetModel(hp=ProphetModel({}).default_hp)
    with pytest.raises(ModelFitError):
        m.fit(df, ['date', 'qty'])
```

03 Boundary Value Cases

Parameters at or near their valid extremes — the most common source of off-by-one errors

BV · Boundary Values

These cases test smoothing parameters at 0.0 and 1.0, and learning rates at their extremes.

BV-01 calibration_update_rate = 0.0 (Frozen Parameter)

P2 — Medium Risk · Fix before stable release

Affects: TenantParams.update() in tenant_params.py

What happens

A tenant administrator sets calibration_update_rate to 0.0 to freeze a parameter while investigating an anomaly — for example, freezing confidence_base_seasonal while reviewing an unexpected accuracy drop.

Formula

```
new = prior + rate * (evidence - prior)
= prior + 0.0 * (evidence - prior)
= prior # value is unchanged
```

RISK

If the update() formula uses a truthy check on rate (e.g. if rate:) rather than an explicit comparison, a zero rate will be treated as False and the update may be skipped entirely — returning None or raising an error instead of returning prior. The DB UPDATE must still be issued even though the value is unchanged (idempotent write).

EXPECTED BEHAVIOUR

update() must return prior exactly. The DB UPDATE is still issued (idempotent). get() after update() must still return the original value unchanged.

Test Sketch

```
def test_update_with_zero_rate_leaves_value_unchanged():
    tp = TenantParams('t1', {
        'confidence_base_cold_start': Decimal('0.50'),
        'calibration_update_rate': Decimal('0.00'),
    })
    conn = FakeConn()
    new_value = tp.update('confidence_base_cold_start', 0.80, conn)
    assert new_value == pytest.approx(0.50)
    assert tp.get('confidence_base_cold_start') == pytest.approx(0.50)
```

BV-02 calibration_update_rate = 1.0 (Instant Replacement)

P3 — Low Risk · Fix when convenient

Affects: TenantParams.update() in tenant_params.py

What happens

A rate of 1.0 means instant replacement — the parameter jumps immediately to the evidence value with no memory of the prior. Used when a tenant is being reset after a data correction.

Formula

```
new = prior + 1.0 * (evidence - prior)
= evidence # prior is completely discarded
```

RISK

No crash expected. The risk is correctness: if the formula uses floating-point arithmetic that introduces tiny rounding errors (e.g. 0.7999999 instead of 0.80), the DB write may fail the DECIMAL(3,2) precision constraint.

EXPECTED BEHAVIOUR

update() must return evidence exactly (within float tolerance). The in-memory snapshot must reflect the new value on the next get().

Test Sketch

```
def test_update_with_rate_one_replaces_value():
    tp = TenantParams('t1', {
        'confidence_base_cold_start': Decimal('0.50'),
        'calibration_update_rate': Decimal('1.00'),
    })
```

```

conn = FakeConn()
new_value = tp.update('confidence_base_cold_start', 0.80, conn)
assert new_value == pytest.approx(0.80)
assert tp.get('confidence_base_cold_start') == pytest.approx(0.80)

```

BV-03 TenantParams.update() with Evidence Outside [0, 1]

P3 — Low Risk · Fix when convenient

Affects: TenantParams.update() in tenant_params.py

What happens

Most tenant params are probabilities (fill rate, confidence, threshold) and must stay in [0, 1]. However, evidence is computed by learning handlers from observed data. A calibration bug could produce evidence = 1.05 or evidence = -0.02.

Formula

```

# With prior=0.50, rate=0.90, evidence=2.0:
new = 0.50 + 0.90 * (2.0 - 0.50) = 0.50 + 1.35 = 1.85
# Stores 1.85 in current_value — breaks confidence_ceiling constraint
# DECIMAL(3,2) column rejects values > 9.99 at DB level

```

RISK

A value > 1.0 stored in current_value breaks the confidence_ceiling constraint and will be rejected by the DECIMAL(3,2) column at DB write time. This produces a DB error mid-run rather than a clean validation failure.

EXPECTED BEHAVIOUR

Decision required — choose one option and lock it in before building.

Decision Required — Choose One

Option	Behaviour	Notes
Option A	update() clamps result to [0.0, 1.0] before storing.	<i>Safe. Prevents DB errors. May hide bugs in evidence computation.</i>
Option B	update() raises ValueError('evidence out of range') for values outside [0, 1].	<i>Explicit. Forces callers to validate evidence before calling update().</i>
Option C	update() passes through — callers are responsible for valid evidence.	<i>Simple. Relies on DB constraint as final guard. Produces unclear DB errors.</i>

Test Sketch

```

def test_update_evidence_above_one():
    tp = TenantParams('t1', {
        'confidence_base_cold_start': Decimal('0.50'),
        'calibration_update_rate': Decimal('0.90'),
    })
    conn = FakeConn()
    # Assert based on chosen option:
    new_value = tp.update('confidence_base_cold_start', 2.0, conn)
    assert 0.0 <= new_value <= 1.0 # if Option A (clamping) is chosen

```

BV-04 SESModel with smoothing_level = 0.0

P2 — Medium Risk · Fix before stable release

Affects: SESModel in ses.py

What happens

The HP search space for SES is [0.1, 0.2, 0.3, 0.4, 0.5], so 0.0 cannot arrive via Thompson Sampling. However, a misconfigured overrides_dict in seed_tenant_params() could introduce it. smoothing_level=0.0 means the model never updates from its initial level estimate — it ignores all observations.

RISK

EXPECTED BEHAVIOUR

statsmodels SimpleExpSmoothing with smoothing_level=0.0 and optimized=False may raise ValueError about degenerate parameters. If the numpy fallback path is not triggered, an unhandled exception propagates out of fit().

Must not propagate an unhandled exception. _level must be finite. With alpha=0.0, the numpy fallback path sets level equal to the series mean (since a zero learning rate produces a constant equal to the initial level, which the numpy path initialises as the mean).

Test Sketch

```
def test_ses_alpha_zero_does_not_crash(df_normal):
    m = SESModel(hp={'smoothing_level': 0.0})
    m.fit(df_normal, ['date', 'qty'])
    assert np.isfinite(m._level)
    result = m.predict(df_normal, ['date', 'qty'], horizon=30)
    assert np.all(np.isfinite(result))
    assert np.all(result >= 0)
```

BV-05 SESModel / HoltLinearTrend with smoothing_level = 1.0

P2 — Medium Risk · Fix before stable release

Affects: SESModel in ses.py, HoltLinearTrend in holt.py

What happens

smoothing_level=1.0 means the model has no memory — it always equals the most recent observation. For HoltLinearTrend this also applies to smoothing_trend=1.0, meaning the trend is always the last single-period increment.

RISK

statsmodels with optimized=False and smoothing_level=1.0 can produce a level equal to the last series value only, then a trend equal to the last single-period increment. For a volatile series this can produce extreme 365d forecasts. The damped_trend mechanism should limit this for Holt, but has not been verified at the smoothing_level=1.0 boundary specifically.

EXPECTED BEHAVIOUR

Both models must complete fit() and predict_all_horizons() without crashing. _level must be finite. All horizon values must be finite and non-negative.

Test Sketch

```
@pytest.mark.parametrize('alpha', [0.0, 1.0])
def test_ses_at_extreme_alpha_does_not_crash(alpha, df_normal):
    m = SESModel(hp={'smoothing_level': alpha})
    m.fit(df_normal, ['date', 'qty'])
    assert np.isfinite(m._level)
    result = m.predict(df_normal, ['date', 'qty'], horizon=30)
    assert np.all(np.isfinite(result))
    assert np.all(result >= 0)
```

BV-06 HoltLinearTrend with smoothing_trend = 0.0 (Frozen Slope)

P2 — Medium Risk · Fix before stable release

Affects: HoltLinearTrend in holt.py

What happens

smoothing_trend=0.0 freezes the trend component at its initial estimate and never updates it. The HP search space minimum is 0.05, but a misconfigured seed could introduce it. The initial trend is computed as (series[-1] - series[0]) / (n - 1). On a declining or flat series this can be zero or slightly negative.

RISK

A frozen negative trend with damped_trend=False will produce negative forecasts at long horizons — e.g. level=10, trend=-0.5, at t=30: forecast = 10 - 15 = -5. If the clamp in predict() is applied only to the final array (not intermediate cumulative sums), the cumulative sum passed to bootstrap_quantiles() will be negative, breaking the p50 ≤ p80 ≤ p90 ordering invariant.

EXPECTED BEHAVIOUR

predict() must clamp output to ≥ 0 even with a frozen negative trend. All horizon values in predict_all_horizons() must be ≥ 0. The clamp must be applied before passing to bootstrap_quantiles().

Test Sketch

```
def test_holt_zero_smoothing_trend_does_not_produce_negatives(df_trending):
    m = HoltLinearTrend(hp={
        'smoothing_level': 0.3, 'smoothing_trend': 0.0, 'damped_trend': False
```

```
})  
m.fit(df_trending, ['date', 'qty'])  
result = m.predict(df_trending, ['date', 'qty'], horizon=365)  
assert np.all(result >= 0), 'Holt must clamp to zero even with frozen trend'
```


04 Business Logic Cases

Domain-specific scenarios that represent real product lifecycle edge cases

BL · Business Logic

Real product behaviours: declining SKUs, extinction demand, feature reliability failures.

BL-01 Declining Demand in HoltLinearTrend

P1 — Highest Risk · Fix before production

Affects: HoltLinearTrend.predict(), predict_all_horizons() in holt.py

What happens

A product approaching end-of-life — a seasonal item past peak, a discontinued SKU, or a product being replaced by a newer model — shows steadily declining demand. Holt fits a negative trend in this case.

Formula

```
# With damped_trend=False and trend=-0.5:
forecast[t] = level + trend * t
forecast[30] = 10 + (-0.5 * 30) = 10 - 15 = -5.0 # WRONG — must be clamped to 0

# Cumulative sum for predict_all_horizons():
# If individual day values are clamped AFTER summing, the sum is already negative.
# Clamp must be applied PER DAY, not on the final sum.
```

RISK

If the clamp is applied only to the final array and not per-day before cumulative summing, the cumulative sum passed to bootstrap_quantiles() is negative. bootstrap_quantiles() adds residuals to this negative point — samples are clamped to zero but the point itself remains negative, producing degenerate p50/p80/p90 where all quantiles equal zero regardless of actual uncertainty.

EXPECTED BEHAVIOUR

predict() must clamp each daily value to ≥ 0 before returning. predict_all_horizons() must clamp the cumulative sum to ≥ 0 before calling bootstrap_quantiles(). Both with damped_trend=True and damped_trend=False must produce non-negative output.

Test Sketch

```
@pytest.fixture
def df_declining():
    return pd.DataFrame({
        'date': pd.date_range('2024-01-01', periods=90),
        'qty': np.maximum(50 - np.arange(90, dtype=float) * 0.5, 0),
    })

@pytest.mark.parametrize('damped', [True, False])
def test_holt_declining_series_no_negatives(damped, df_declining):
    m = HoltLinearTrend(hp={
        'smoothing_level': 0.3, 'smoothing_trend': 0.1, 'damped_trend': damped})
    m.fit(df_declining, ['date', 'qty'])
    assert m._trend < 0, 'Expected negative trend on declining series'
    result = m.predict(df_declining, ['date', 'qty'], horizon=365)
    assert np.all(result >= 0), 'predict() must clamp declining forecast to zero'

def test_holt_declining_all_horizons_non_negative(df_declining):
    m = HoltLinearTrend(hp=HoltLinearTrend({}).default_hp)
    m.fit(df_declining, ['date', 'qty'])
    rh = m.predict_all_horizons(df_declining, ['date', 'qty'])
    for key, val in rh.items():
        for q, v in val.items():
            assert v >= 0, f'{key}.{q} = {v} (negative on declining series)'
```

BL-02 Croston TSB — Extinction Pattern

P2 — Medium Risk · Fix before stable release

Affects: CrostonMethod._fit_tsb() in croston.py

What happens

TSB (Teunter-Syntetos-Babai) is specifically designed for products that may stop selling permanently. The defining scenario: early demand events followed by a long run of zeros as the product dies off. Example: 5 demand events in the first 20 days, then

40 consecutive days of zero demand.

Formula

```
# TSB tracks p_t (demand probability), which decays on every zero period:
p_t = alpha * 0 + (1 - alpha) * p_(t-1)
= (1 - alpha)^k * p_initial

# After 40 zero periods with alpha=0.10:
p = (0.90)^40 ≈ 0.015

# daily_rate = demand_size * p ≈ some_demand_size * 0.015 (near-zero)
```

RISK	EXPECTED BEHAVIOUR
If the TSB extinction-decay logic is not correctly implemented, TSB and classic Croston will produce the same forecast on this extinction data. This means TSB is not actually tracking demand probability decay — it is computing the same rate as classic, which is wrong for dying products.	TSB forecast_365d must be meaningfully lower than classic Croston on the same extinction series. After 40 zero periods, daily_rate should be near-zero. TSB _daily_rate must be less than classic _daily_rate on this specific series.

Test Sketch

```
@pytest.fixture
def df_extinction():
    qty = np.zeros(60)
    qty[[2, 5, 9, 14, 18]] = [8.0, 6.0, 10.0, 7.0, 5.0]
    return pd.DataFrame({
        'date': pd.date_range('2024-01-01', periods=60), 'qty': qty,
    })

def test_tsbProducesLowerForecastThanClassic(df_extinction):
    m_tsb = CrostonMethod(hp={'alpha': 0.10, 'interval_type': 'TSB'})
    m_classic = CrostonMethod(hp={'alpha': 0.10, 'interval_type': 'classic'})
    m_tsb.fit(df_extinction, ['date', 'qty'])
    m_classic.fit(df_extinction, ['date', 'qty'])
    assert m_tsb._daily_rate < m_classic._daily_rate, (
        f'TSB rate {m_tsb._daily_rate:.4f} should be lower than '
        f'classic {m_classic._daily_rate:.4f} on extinction series'
    )

def test_tsbDailyRateNearZeroAfterExtinction(df_extinction):
    m = CrostonMethod(hp={'alpha': 0.10, 'interval_type': 'TSB'})
    m.fit(df_extinction, ['date', 'qty'])
    assert m._daily_rate < 1.0, 'TSB rate should be near-zero after long silence'
```

BL-03 Sub-Stage 9.2 — All Optional Features Fail Reliability Floor

P2 — Medium Risk · Fix before stable release Affects: run_feature_engineering() in sub_stage_92.py

What happens

A new tenant has no feature history yet, so feature_reliability returns 0.0 for every optional feature. Every candidate feature is dropped at Step 1 (reliability filter), leaving only ['date', 'qty']. Step 4 (additive feature search) then receives an empty candidate set.

RISK	EXPECTED BEHAVIOUR
Step 4 iterates over the candidate feature list. If it uses any() or next() on an empty list, it raises StopIteration or returns incorrect results. The BatchWriter row must still be written with features_used = ['date', 'qty'] and improved_mape equal to baseline_mape (no improvement since no features were tried).	result.selected_features must equal ['date', 'qty']. A feature_decisions_s9 row must still be written to BatchWriter. improved_mape must equal baseline_mape (no change). No crash from empty candidate list in Step 4.

Test Sketch

```
def test_all_optional_features_fail_reliability(df_normal, mock_ctx, mock_params, mock_bw):
    df = df_normal.copy()
    df['promo_flag'] = 0.0
    df['day_of_week'] = df['date'].dt.dayofweek.astype(float)
    model = SESModel(hp=SESModel({}).default_hp)
    preloaded = {
```

```
'feature_reliability': {
mock_ctx.sku_id: {'promo_flag': 0.05, 'day_of_week': 0.10}
}, # both below floor of 0.30
'promo_decisions': {}, 'feature_history': {},
}
result = run_feature_engineering(
mock_ctx, df, model, preloaded, mock_params, mock_bw)
assert result.selected_features == ['date', 'qty']
assert 'feature_decisions_s9' in mock_bw.tables
```

05 Error Handling Cases

Failure propagation paths — the system must fail loudly and cleanly, never silently

EH · Error Handling

When things break, state must transition to FAILED, locks must be released, and exceptions must propagate.

EH-01 All HP Configs Raise ModelFitError in Sub-Stage 9.3

P1 — Highest Risk · Fix before production

Affects: run_hp_tuning() in sub_stage_93.py

What happens

A model class has a bug that causes every configuration in its HP search space to raise ModelFitError — for example, a data type incompatibility introduced by an upstream schema change, or a transient numerical instability on a specific SKU's data. The existing test test_model_fit_error_assigns_mape_1 covers one config failing. This verifies all 9 configs failing.

RISK

run_hp_tuning() may re-raise after all configs fail, crashing the run for this SKU and potentially others if the exception is not isolated. best_hp may be set to None rather than default_hp, causing a downstream AttributeError. early_stopped may incorrectly be True (early stop requires a successful eval, not failure). The BatchWriter row may not be written.

EXPECTED BEHAVIOUR

run_hp_tuning() must not re-raise after all configs fail. best_hp must fall back to default_hp (not None). validation_mape must be 1.0 (the failure sentinel). early_stopped must be False. A hyperparameter_decisions row must still be written to BatchWriter.

Test Sketch

```
def test_all_configs_fail_returns_default_hp(df_normal, mock_ctx, mock_params, mock_bw):
    class AlwaysFails(NaiveForecast):
        def fit(self, df, features, sample_weights=None):
            raise ModelFitError('every config fails')
    model = AlwaysFails(hp=NaiveForecast({}).default_hp)
    result = run_hp_tuning(mock_ctx, df_normal, model, {}, mock_params, mock_bw)
    assert result.best_hp == model.default_hp
    assert result.validation_mape == 1.0
    assert result.early_stopped is False
    assert 'hyperparameter_decisions' in mock_bw.tables
    row = mock_bw.row_for('hyperparameter_decisions')
    assert row['hyperparameters'] == model.default_hp
```

EH-02 Bare Exception from Handler Crashes run()

P1 — Highest Risk · Fix before production

Affects: run() in state_machine.py

What happens

A handler encounters an unexpected error that is not a Stage9Error subclass — for example, a ConnectionResetError from a dropped DB connection mid-handler, a KeyError from a missing key in the preloaded dict, or an AttributeError from a None value in the context object.

RISK

If run() only catches Stage9Error and re-raises others without transitioning to FAILED, the state log will show the last successful state (e.g. PERCEIVING) and never reach FAILED. A monitoring query for stuck-in-PERCEIVING runs will not alert correctly. The Redis lock may also be left dangling if the finally block is not reached.

EXPECTED BEHAVIOUR

run() must catch all exceptions (not just Stage9Error). Transition state to FAILED and write to agent_state_log_s9. Release the Redis lock in the finally block — even if the exception occurred before the lock was acquired, the finally block must handle token=None gracefully. Re-raise the original exception so the caller can log it.

Test Sketch

```
def test_bare_exception_transitions_to_failed():
    db = FakeDB()
    fake_redis = _make_fake_redis()
    with patch('state_machine.handlers.preloading.handler',
               side_effect=RuntimeError('disk full')), \
         patch.object(run_lock_module, '_get_client', return_value=fake_redis):
```

```

with pytest.raises(RuntimeError, match='disk full'):
    run(tenant_id='t-bare-exc', run_id='r-001', db=db)
states = [row['new_state'] for row in db.written]
assert AgentState.FAILED.value in states

def test_bare_exception_releases_redis_lock():
    db = FakeDB()
    fake_redis = _make_fake_redis()
    with patch('state_machine.handlers.preloading.handler',
               side_effect=RuntimeError('oom')), \
        patch.object(run_lock_module, '_get_client', return_value=fake_redis):
        with pytest.raises(RuntimeError):
            run(tenant_id='t-lock-release', run_id='r-002', db=db)
    lock = RedisRunLock()
    acquired, token = lock.acquire('t-lock-release')
    assert acquired, 'Lock must be released after bare Exception'
    lock.release('t-lock-release', token)

```

EH-03 TenantParams.load() Returns 0 Rows

P2 — Medium Risk · Fix before stable release

Affects: TenantParams.load() in tenant_params.py, PERCEIVING handler

What happens

A new tenant was added to the tenants table but seed_tenant_params() was never called (or was rolled back). When run() reaches the PERCEIVING handler and calls TenantParams.load(), the SELECT returns 0 rows.

RISK	EXPECTED BEHAVIOUR
If TenantParams.load() raises on 0 rows, the exception type matters: an unexpected exception type here will bypass the Stage9Error handler in run() and leave the run in an unclear state. If load() returns an object with len=0 but get() then raises KeyError (not a known Stage9Error), the same problem occurs.	TenantParams.load() must return a TenantParams object with len(tp) == 0 — not raise. The PERCEIVING handler detects len(tp) == 0 and raises TenantParamNotFoundError. run() catches TenantParamNotFoundError, transitions to FAILED, and re-raises. The caller receives TenantParamNotFoundError and can trigger an automatic seeding retry.

Test Sketch

```

def test_load_with_zero_rows_returns_empty_tenant_params():
    conn = FakeConn(select_rows=[])
    tp = TenantParams.load(str(uuid.uuid4()), conn)
    assert len(tp) == 0

def test_get_on_empty_tenant_params_raises_unknown_param_error():
    tp = TenantParams('t1', {})
    with pytest.raises(UnknownParamError):
        tp.get('confidence_floor')

```

06 Seed & DB Boundary Cases

Parameter storage layer — boundary values in override dicts and DB column constraints

SD · Seed & DB	These cases test <code>seed_tenant_params()</code> and the DB layer, focusing on zero overrides and idempotency.
---------------------------	--

SD-01	seed_tenant_params() Override with Value 0.0
--------------	---

P2 — Medium Risk · Fix before stable release	Affects: seed_tenant_params() in seed.py
---	---

What happens

An administrator explicitly sets a parameter to 0.0 via the override dict — for example, disabling promo weighting by setting `service_level_target=0.0` during a data-quality incident.

RISK	EXPECTED BEHAVIOUR
Python's truthiness: if <code>override_val</code> : evaluates 0.0 as False. If <code>seed_tenant_params()</code> checks whether an override exists with a truthy check rather than if <code>override_val</code> is not None:, the zero value is silently dropped and the default is seeded instead. The administrator believes they zeroed out the parameter but finds the default in the DB with no error raised.	The zero override must be stored in the DB as 0.0. The check for an override's presence must use 'is not None', not a truthy check. No error raised.

Test Sketch

<pre>def test_override_zero_is_applied_not_dropped(): conn = FakeConn() seed_tenant_params(str(uuid.uuid4()), 'established', overrides_dict={'service_level_target': 0.0}, conn=conn,) by_name = {params[1]: params[2] for _, params in conn.cur.executed} assert by_name['service_level_target'] == Decimal('0.0'), ('A zero override must not be silently replaced by the default')</pre>

SD-02 · seed_tenant_params() Override with Value > 1.0

Priority	Affects
P3	seed_tenant_params() in seed.py

Some params are not probabilities — `exploit_threshold_new` is seeded at 8.00. But `service_level_target` is a fill-rate target (max 1.0). An override of 1.5 is likely a data-entry error. The current code accepts any Decimal-convertible float with no range validation.

Option	Behaviour	Status
Option A	Accept any numeric override — callers are responsible for valid ranges.	No validation in seed.py. DB column constraint is last defence.
Option B	Validate params documented as [0,1] in <code>TENANT_LEARNING_PARAMS_DEFAULTS</code> . Raise <code>ValueError</code> for out-of-range.	Explicit. Forces callers to validate. More engineering work.

Document the chosen option and add a test to lock it in. The test sketch is in `EDGE_CASES_SEED_DB.md`.